

Materiał 01 - ExpressJS i pierwsze REST API

Cel

W tym materiale zbudujemy pierwszą aplikację backendową wykorzystującą **Node.js**, **ExpressJS** oraz architekturę **REST API**.

Po jego przejściu powinieneś rozumieć:

- czym jest backend i REST API,
- czym różni się request od response,
- jak działają podstawowe HTTP methods,
- jak tworzyć endpoints w ExpressJS,
- jak odbierać dane z URL oraz request body,
- jak zwracać poprawne HTTP status codes,
- jak wykonywać podstawową validation,
- czym różni się zwykły CRUD od prostej business logic.

Na tym etapie **nie korzystamy jeszcze z bazy danych**. Dane będziemy przechowywać tymczasowo w pamięci aplikacji.

Dzięki temu skupimy się najpierw na mechanizmie działania API.

1. Co właściwie budujemy?

W klasycznej aplikacji webowej użytkownik nie komunikuje się bezpośrednio z bazą danych.

Najczęściej wygląda to mniej więcej tak:

```
User
|
v
Frontend
```

```
|  
| HTTP Request  
v  
REST API  
|  
v  
Backend  
|  
v  
Database
```

Frontend może być napisany np. w Angularze.

Backend może być napisany np. w ExpressJS.

Baza danych może być np. PostgreSQL.

W tym materiale interesuje nas przede wszystkim fragment:

```
Client -> HTTP Request -> ExpressJS -> HTTP Response -> Client
```

Klientem nie musi być jeszcze Angular.

Do testowania API możemy wykorzystać:

- przeglądarkę,
- Postman,
- Insomnia,
- pliki `.http`,
- `curl`.

2. Request i response

Każda komunikacja z naszym API zaczyna się od **requestu**.

Przykład:

```
GET /api/tasks
```

Klient mówi tutaj:

 Chcę pobrać listę zadań.

Backend odbiera request, wykonuje odpowiedni kod i zwraca **response**.

Przykładowy response:

```
[
  {
    "id": 1,
    "title": "Prepare presentation",
    "completed": false
  },
  {
    "id": 2,
    "title": "Learn Express",
    "completed": true
  }
]
```

Response składa się jednak nie tylko z danych.

Bardzo ważnym elementem jest również **HTTP status code**.

Przykład:

```
200 OK
```

oznacza:

 Request został poprawnie obsłużony.

3. HTTP methods

REST API wykorzystuje HTTP methods do określenia, **co chcemy zrobić z danym zasobem**.

Dla zasobu `tasks` możemy mieć:

Method	Endpoint	Znaczenie
GET	<code>/api/tasks</code>	pobranie wszystkich zadań
GET	<code>/api/tasks/1</code>	pobranie zadania o ID 1
POST	<code>/api/tasks</code>	utworzenie nowego zadania
PATCH	<code>/api/tasks/1</code>	częściowa aktualizacja zadania
DELETE	<code>/api/tasks/1</code>	usunięcie zadania

Zwróć uwagę, że w URL używamy rzeczowników:

```
/api/tasks
```

a nie:

```
/api/getTasks  
/api/createTask  
/api/deleteTask
```

Informację o operacji przekazuje już HTTP method.

4. Pierwsza aplikacja ExpressJS

Tworzymy nowy projekt:

```
mkdir first-api
cd first-api

npm init -y
npm install express
```

Następnie tworzymy plik:

```
server.js
```

Najprostsza aplikacja Express wygląda tak:

```
const express = require("express");

const app = express();

app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

Uruchamiamy:

```
node server.js
```

Mamy działający server, ale jeszcze nie obsługuje żadnego endpointu.

5. Pierwszy endpoint

Dodajmy:

```
app.get("/api/tasks", (req, res) => {  
  res.json([  
    {  
      id: 1,  
      title: "Learn Express",  
      completed: false  
    }  
  ]);  
});
```

Teraz request:

```
GET http://localhost:3000/api/tasks
```

zwróci:

```
[  
  {  
    "id": 1,  
    "title": "Learn Express",  
    "completed": false  
  }  
]
```

Zwróć uwagę na dwa obiekty:

```
req  
res
```

`req` oznacza **request**.

Znajdziemy w nim informacje przesłane przez klienta.

`res` oznacza **response**.

Używamy go do przygotowania odpowiedzi.

6. Dane przechowywane w pamięci

Na razie nie mamy database.

Możemy więc stworzyć zwykłą tablicę:

```
let tasks = [  
  {  
    id: 1,  
    title: "Prepare presentation",  
    completed: false  
  },  
  {  
    id: 2,  
    title: "Learn Express",  
    completed: true  
  }  
];
```

Endpoint może teraz zwrócić tę tablicę:

```
app.get("/api/tasks", (req, res) => {  
  res.json(tasks);  
});
```

Ważne

Te dane istnieją tylko podczas działania aplikacji.

Jeżeli zatrzymamy server i uruchomimy go ponownie, wszystkie zmiany zostaną utracone.

Później zastąpimy tę tablicę prawdziwą database.

7. Path parameters

Co jeżeli chcemy pobrać tylko jedno zadanie?

Możemy utworzyć endpoint:

```
GET /api/tasks/:id
```

Fragment:

```
:id
```

oznacza dynamiczną część URL.

Przykład:

```
GET /api/tasks/2
```

W ExpressJS możemy pobrać tę wartość przez:

```
req.params
```

Przykład:

```
app.get("/api/tasks/:id", (req, res) => {  
  const id = Number(req.params.id);  
  
  const task = tasks.find(task => task.id === id);  
  
  res.json(task);  
});
```

Ale mamy tutaj problem.

Co się stanie dla:

```
GET /api/tasks/999
```

Takiego zadania nie ma.

Nie powinniśmy wtedy zwracać:

```
200 OK
```

Lepszym rozwiązaniem jest:

```
app.get("/api/tasks/:id", (req, res) => {
  const id = Number(req.params.id);

  const task = tasks.find(task => task.id === id);

  if (!task) {
    return res.status(404).json({
      message: "Task not found"
    });
  }

  res.json(task);
});
```

Teraz klient otrzyma:

404 Not Found

oraz:

```
{
  "message": "Task not found"
}
```

8. Najważniejsze HTTP status codes

Na początek wystarczy kilka:

Code	Znaczenie
200	request wykonany poprawnie
201	utworzono nowy zasób
204	operacja poprawna, ale bez response body
400	błędne dane wejściowe
404	zasób nie istnieje
409	konflikt z aktualnym stanem systemu
500	nieoczekiwany błąd backendu

Status code jest częścią API.

Nie powinniśmy zawsze zwracać 200, tylko dlatego, że server się nie wysypał.

9. Request body

Teraz chcemy umożliwić utworzenie nowego zadania.

Request może wyglądać tak:

```
POST /api/tasks
Content-Type: application/json
```

Body:

```
{
  "title": "Learn REST API"
```

```
}
```

Żeby ExpressJS potrafił odczytać JSON z requestu, dodajemy:

```
app.use(express.json());
```

Następnie:

```
app.post("/api/tasks", (req, res) => {  
  console.log(req.body);  
  
  res.json(req.body);  
});
```

Dane przesłane przez klienta są dostępne jako:

```
req.body
```

10. Tworzenie nowego zasobu

Możemy teraz stworzyć nowy obiekt:

```
app.post("/api/tasks", (req, res) => {  
  const task = {  
    id: tasks.length + 1,  
    title: req.body.title,  
    completed: false  
  };  
});
```

```
tasks.push(task);

res.status(201).json(task);
});
```

Request:

```
{
  "title": "Learn REST API"
}
```

Response:

```
{
  "id": 3,
  "title": "Learn REST API",
  "completed": false
}
```

Status:

```
201 Created
```

11. Validation

Obecny endpoint pozwala jednak wysłać:

```
{}
```

albo:

```
{  
  "title": ""  
}
```

i mimo tego utworzy zadanie.

To nie jest dobre API.

Backend powinien sprawdzić dane wejściowe.

Przykład:

```
app.post("/api/tasks", (req, res) => {  
  const { title } = req.body;  
  
  if (!title || title.trim().length < 3) {  
    return res.status(400).json({  
      message: "Title must contain at least 3 characters"  
    });  
  }  
  
  const task = {  
    id: tasks.length + 1,  
    title: title.trim(),  
    completed: false  
  };  
  
  tasks.push(task);  
  
  res.status(201).json(task);  
});
```

Teraz request:

```
{  
  "title": ""  
}
```

zakończy się:

```
400 Bad Request
```

12. Query parameters

Czasami chcemy pobrać tylko część danych.

Przykład:

```
GET /api/tasks?completed=true
```

Fragment:

```
?completed=true
```

to **query parameter**.

W ExpressJS znajdziemy go w:

```
req.query
```

Możemy więc napisać:

```
app.get("/api/tasks", (req, res) => {
  const { completed } = req.query;

  if (completed === undefined) {
    return res.json(tasks);
  }

  const value = completed === "true";

  const result = tasks.filter(task => {
    return task.completed === value;
  });

  res.json(result);
});
```

13. Aktualizacja danych

Założmy, że chcemy oznaczyć zadanie jako wykonane.

Możemy wykorzystać:

```
PATCH /api/tasks/:id
```

Przykładowy request:

```
{
  "completed": true
}
```


Implementacja:

```
app.patch("/api/tasks/:id", (req, res) => {
  const id = Number(req.params.id);

  const task = tasks.find(task => task.id === id);

  if (!task) {
    return res.status(404).json({
      message: "Task not found"
    });
  }

  if (typeof req.body.completed !== "boolean") {
    return res.status(400).json({
      message: "Completed must be boolean"
    });
  }

  task.completed = req.body.completed;

  res.json(task);
});
```

14. Usuwanie zasobu

Do usunięcia możemy wykorzystać:

```
DELETE /api/tasks/:id
```

Przykład:

```
app.delete("/api/tasks/:id", (req, res) => {
  const id = Number(req.params.id);

  const index = tasks.findIndex(task => task.id === id);

  if (index === -1) {
    return res.status(404).json({
      message: "Task not found"
    });
  }

  tasks.splice(index, 1);

  res.status(204).send();
});
```

204 No Content oznacza, że operacja zakończyła się powodzeniem, ale response nie zawiera body.

15. CRUD a business logic

Do tej pory wykonujemy głównie CRUD:

- Create
- Read
- Update
- Delete

CRUD jest potrzebny, ale sam CRUD nie opisuje jeszcze całej logiki aplikacji.

Wyobraźmy sobie system rezerwacji sal.

Mamy endpoint:

POST /api/reservations

i request:

```
{
  "roomId": 3,
  "start": "2026-10-10T10:00",
  "end": "2026-10-10T12:00"
}
```

Technicznie możemy taki obiekt dodać do tablicy.

Ale przed jego zapisaniem system powinien sprawdzić:

| Czy ta sala nie jest już zarezerwowana w tym czasie?

To jest **business rule**.

Możemy mieć więc poprawny JSON i istniejącą salę, ale nadal odmówić wykonania operacji.

Przykładowo:

409 Conflict

```
{
  "message": "Room is already reserved in this time range"
}
```

To właśnie zaczyna odróżniać prawdziwą aplikację od zwykłego CRUD-a.

16. Nie ufamy klientowi

Jedna z najważniejszych zasad backendu:

| Dane przesłane przez klienta zawsze wymagają sprawdzenia.

Frontend może mieć piękny formularz z validation.

Nie oznacza to jednak, że backend może założyć, że dane zawsze będą poprawne.

API może zostać wywołane bez frontendu, np. przez:

- Postman,
- `curl`,
- inny frontend,
- skrypt,
- inną aplikację.

Dlatego validation oraz business rules muszą być egzekwowane również po stronie backendu.

17. Podsumowanie

Po tym materiale powinieneś rozumieć następujący przepływ:

```
Client
  |
  | HTTP Request
  v
Express route
  |
  | req.params
  | req.query
  | req.body
```

```
  v
Validation
  |
  v
Business Logic
  |
  v
Data
  |
  v
HTTP Response
```

Najważniejsze elementy:

```
req.params -> dane z URL

req.query -> query parameters

req.body -> dane przesłane w body

res.json() -> JSON response

res.status() -> HTTP status code
```

Najważniejsza rzecz:

API nie polega tylko na tym, żeby endpoint "działał".

Powinien również:

- poprawnie obsługiwać błędne dane,
- zwracać odpowiednie HTTP status codes,
- reagować na brak zasobu,
- pilnować zasad działania systemu.